

Reviewer Pack — Sidon B_2 -Witness Failure of $\text{ACC}^0[2]$ for Sipser via MOD-Gate...

Entry b6fc094b0a25 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 17:00:17 UTC

Sidon B_2 -Witness Failure of $\text{ACC}^0[2]$ for Sipser via MOD-Gate Outputs

Entry ID: b6fc094b0a25 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 17:00:17 UTC

1. Conjecture

Field A (mathematical branch): Sidon / B_2 -set additive combinatorics — the Erdős–Turán maximum representation function $r(A) := \max_{w \in A+A} \#\{(a,a') \in A^2 : a + a' = w\}$, classifying $A \subseteq F_2^N$ as a Sidon (B_2) set iff $r(A) = 2$ (only diagonal pairs), with the ‘Sidon defect’ $\text{sd}(A) := r(A) - 2 \in [0, |A|-2]$ measuring deviation from B_2 -ness (Erdős–Turán 1941 ‘On a problem of Sidon in additive number theory’; Lindström 1969 ‘An inequality for B_2 -sequences’; Cilleruelo 2010; O’Bryant 2004 survey ‘A complete annotated bibliography of work related to Sidon sets and B_h sets’). A non-ring max-of-integer-counts functional: uses max + integer counting + F_2 -XOR; the OUTER max is sup-style and no F_q polynomial extension preserves max-representation count under ring lifts, so the invariant is Aaronson–Wigderson algebrization-safe. Distinct from blacklisted Plünnecke doubling $|A+A|/|A|$ on truth-table top-Fourier supports (a sumset CARDINALITY functional), Chang η -spectrum cardinality on wire-source labels (a Fourier-spectrum functional), additive energy $E(A) = \sum_w r(A,w)^2$ on out-degree sequences (an L^2 functional in $\mathbb{Z}$, not max in F_2^N), Lehmer-pair close-zero spacings, and Bourgain–Tzafriri sub-column dispersion; r is the MAX (not sum, not size) of the representation function on F_2^N gate-output vectors. ArXiv ‘Sidon set’ AND (‘ ACC^0 ’ OR ‘lifting theorem’ OR ‘circuit lower bound’) returns 0 direct hits with <5 adjacent papers (Behrend–Roth, Ruzsa–Plünnecke), none on ACC^0 MOD-gate output multisets. **Field B** (complexity object): $\text{ACC}^0[2]$ circuit size $s(C)$ and depth $d(C)$ for the depth- d Sipser–Yao function $S_{\{n,d\}}$ (alternating AND/OR tree of fan-in $n^{\{1/d\}}$), in the Williams 2014 $\text{NEXP} \not\subseteq \text{ACC}^0$ / Murray–Williams 2018 algorithmic-method regime targeting explicit ACC^0 lower bounds. Bridged to LIFTING via ‘within-circuit gate-output lifting’ in the Raz–McKenzie / Göös–Pitassi–Watson tradition: each MOD-2 gate g is an F_2 -linear functional of its inputs, and its truth-table on $N=1024$ seed-fixed random inputs lifts the circuit DAG into an additive-combinatorial object $V_C \subseteq F_2^N$. The invariant sd is STRUCTURAL on the circuit wiring (two $\text{ACC}^0[2]$ circuits with identical Boolean function can have different sd), shielding from Razborov–Rudich natural proofs; the within-circuit lifting plays the role of the standard query-to-communication gadget composition.

Statement:

Let C be an $\text{ACC}^0[2]$ circuit on n inputs of depth d with m_C MOD-2 gates of fan-in 4 whose output vectors are F_2 -linearly independent. Fix $N=1024$ uniformly random inputs (seed s), let $v_g \in F_2^N$ be gate g ’s output vector, and $V_C(s) := \{v_g : g \text{ a MOD-2 gate of } C\}$. Define $r(C,s) := \max_{w \in F_2^N} \#\{(a,b) \in V_C(s)^2 : a \text{ XOR } b = w\}$ and $\text{sd}(C) := \text{median over 30 seeds of } (r(C,s) - 2)$. Conjecture (B_2 -Witness for $\text{ACC}^0[2]$): for every such $\text{ACC}^0[2]$ circuit C with $m_C \geq 8$ and $d \leq 5$, $\text{sd}(C) \leq \lceil \sqrt{m_C} \rceil$, i.e., MOD-2

gate outputs are approximately-Sidon under typical input subsamples. A single $(C, \text{seed-batch})$ with $\text{sd}(C) > \lceil \sqrt{m_C} \rceil$ refutes the conjecture; corollary: by Razborov-Smolensky, the Sipser function $S_{\{n,d\}}$ forces a non-random F_2 -polynomial structure in V_C breaking the bound, separating $S_{\{n,d\}}$ from depth- d $\text{ACC}^0[2]$.

Rationale (proposer’s reasoning):

$\text{ACC}^0[2]$ circuits of bounded depth compute low-degree F_2 -polynomial approximations of their inputs (Razborov 1987; Smolensky 1987), so MOD_2 gate outputs span an F_2 -vector subspace of dimension $\Theta(n^{\{O(d)\}})$ inside F_2^N which, for $m_C \leq \sqrt{(\dim)}$, behaves like a random Sidon set with r^* concentrating at $\bar{2}$. The Sipser function’s depth- d alternation forbids such randomness: any $\text{ACC}^0[2]$ computation of $S_{\{n,d\}}$ must inject correlations that drive r^* above $\sqrt{m_C}$, yielding an ACC^0 obstruction. The conjecture is shielded from Razborov-Rudich (sd is wiring-dependent, not truth-table dependent), from Aaronson-Wigderson ($\text{max} + \text{count}$ are non-ring), and instantiates the LIFTING focus by treating MOD -gate output vectors as a gadget-style algebraic lift of the underlying decision-tree structure.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e6b25c40f23eb66

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 3 depths $\{2,3,4\} \times$ 3 $m_C \{8,16,24\} \times$ 3 $n \{16,24,32\} = 810$ $\text{ACC}^0[2]$ circuits with F_2 -independent MOD_2 outputs on $N=1024$ inputs, in every (d,m_C,n) cell the median over 30 seeds of $(r^*(C,s)-2)$ must be $\leq \lceil \sqrt{m_C} \rceil$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Sidon set $B_2 F_2$ representation function circuit lower bound - ACC^0 MOD_2 gate output additive combinatorics lifting Sipser-Razborov Smolensky XOR gate Sidon additive energy circuit complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
import collections

def matrix_multiply(A, B):
    """Multiply two matrices over F_2."""
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] ^= A[i][k] & B[k][j]
    return result

def gaussian_elimination(matrix):
    """Perform Gaussian elimination over F_2."""
    n = len(matrix)
    if n == 0:
        return matrix
    m = len(matrix[0])
    rank = 0
    for col in range(m):
        pivot = -1
        for row in range(rank, n):
            if matrix[row][col] == 1:
                pivot = row
                break
        if pivot == -1:
            continue
        matrix[rank], matrix[pivot] = matrix[pivot], matrix[rank]
        for row in range(n):
            if row != rank and matrix[row][col] == 1:
                for c in range(col, m):
                    matrix[row][c] ^= matrix[rank][c]
        rank += 1
    return matrix

def is_linearly_independent(vectors):
    """Check if vectors are linearly independent over F_2."""
    if not vectors:
        return True
    n = len(vectors[0])
    matrix = [[0 for _ in range(n)] for _ in range(len(vectors))]
    for i, vec in enumerate(vectors):
        for j, val in enumerate(vec):
            matrix[i][j] = val
    reduced = gaussian_elimination(matrix)
    for i in range(len(reduced)):
        if all(reduced[i][j] == 0 for j in range(len(reduced[i]))):
            return False
    return True
```

```

def build_random_acc0_circuit(n, d, m_C, seed):
    """Build a random ACC0[2] circuit with alternating layers."""
    random.seed(seed)
    layers = []
    for _ in range(d):
        layer_type = random.choice(['AND', 'OR', 'MOD_2'])
        layer = []
        for _ in range(m_C):
            gate_type = random.choice(['AND', 'OR', 'MOD_2']) if layer_type != 'MOD_2' else 'MOD_2'
            fan_in = 4
            inputs = random.sample(range(n), fan_in)
            negations = [random.choice([True, False]) for _ in range(fan_in)]
            layer.append((gate_type, inputs, negations))
        layers.append(layer)
    return layers

def evaluate_circuit(circuit, inputs):
    """Evaluate the circuit on the given inputs."""
    gate_outputs = []
    for layer in circuit:
        layer_outputs = []
        for gate_type, inputs_idx, negations in layer:
            gate_input = [inputs[i] ^ negations[i] for i in inputs_idx]
            if gate_type == 'AND':
                output = all(gate_input)
            elif gate_type == 'OR':
                output = any(gate_input)
            elif gate_type == 'MOD_2':
                output = sum(gate_input) % 2
            layer_outputs.append(output)
        gate_outputs.extend(layer_outputs)
    return gate_outputs

def run_trial(seed):
    """Run one trial of the experiment."""
    random.seed(seed)
    n = random.choice([16, 24, 32])
    d = random.choice([2, 3, 4])
    m_C = random.choice([8, 16, 24])
    N = 1024

    # Build a random ACC0[2] circuit
    circuit = build_random_acc0_circuit(n, d, m_C, seed)

    # Generate random inputs
    inputs = [[random.choice([0, 1]) for _ in range(n)] for _ in range(N)]

    # Evaluate the circuit on the inputs
    gate_outputs = []
    for input_vec in inputs:
        gate_outputs.append(evaluate_circuit(circuit, input_vec))

    # Extract MOD_2 gate outputs
    mod2_outputs = []
    for i in range(len(circuit)):

```

```

    for j in range(len(circuit[i])):
        if circuit[i][j][0] == 'MOD_2':
            mod2_outputs.append([gate_outputs[k][i * len(circuit[i]) + j] for k in range(N)])

# Check linear independence
if not is_linearly_independent(mod2_outputs):
    return {
        "metric_name": "r*(C,s) - 2",
        "metric_value": 0,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

# Compute pairwise XORs and find r*(C,s)
xor_counts = collections.Counter()
for a in mod2_outputs:
    for b in mod2_outputs:
        xor_vec = [a[i] ^ b[i] for i in range(N)]
        xor_counts[tuple(xor_vec)] += 1

r_star = max(xor_counts.values()) if xor_counts else 0
metric_value = r_star - 2
bound = math.ceil(math.sqrt(m_C))

conjecture_holds = metric_value <= bound
counterexample = f"seed={seed}, d={d}, m_C={m_C}, n={n}, r*={r_star}" if not conjecture_holds

return {
    "metric_name": "r*(C,s) - 2",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,

    metric_values = []
    conjecture_holds_counts = 0
    counterexamples = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        metric_values.append(trial["metric_value"])
        if trial["conjecture_holds"]:
            conjecture_holds_counts += 1
        if trial["counterexample"]:
            counterexamples.append(trial["counterexample"])

    mean_metric = sum(metric_values) / len(metric_values) if metric_values else 0
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = conjecture_holds_counts / len(seeds)

```

```

if counterexamples:
    print(f'RESULT: FALSIFIED counterexample="{counterexamples[0]}" first_failing_seed={seeds}')
elif support_fraction >= 0.8:
    print(f'RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}')
else:
    print('RESULT: INCONCLUSIVE reason=insufficient_support')

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66924d2a.py", line 165, in <module>
    trial = run_trial(seed)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66924d2a.py", line 116, in run_trial
    gate_outputs.append(evaluate_circuit(circuit, input_vec))
                        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66924d2a.py", line 88, in evaluate_circuit
    gate_input = [inputs[i] ^ negations[i] for i in inputs_idx]
                  ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in evaluate_circuit before producing any RESULT line, so no data was collected against the pre-registered criterion. | next: Fix the indexing bug in evaluate_circuit (ensure negations array is sized to match the circuit's input fan-in / gate inputs_idx range) and rerun the 810-circuit sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	599816
2	preregistration	claude_max	opus	0	0	8262
3	novelty	claude_max	opus	0	0	5834
4	novelty	claude_max	opus	0	0	6464
5	test_gen	mistral	codestral-latest	0	0	316729
6	test_gen	mistral	codestral-latest	0	0	313550

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	mistral	codestral-latest	0	0	318837
8	test_gen	mistral	codestral-latest	0	0	317545
9	judge	claude_max	opus	0	0	4958

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1891995 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/b6fc094b0a25.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b6fc094b0a25.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b6fc094b0a25.tar.gz` (if generated)